MySQL2之查询操作

查询操作

表结构与数据表的关系

表结构相当于类，面粉模具，表相当于众多对象(一行为一个)，是生产出来的具体面粉、表中的一行数据中的每个字段相当于对象的每个属性

- 类　　表结构相当于类
- 对象　　表相当于众多对象(一行数据为一个对象)
- 属性　　表中的一行数据中的每个字段相当于对象的每个属性

主键

- 特性
  - 唯一性
  - 不为null

  一般情况每个表都会有主键，确保每一行数据都有唯一标识
  没设置mysql底层会自动生成
- 定义　　主键是表中的某一列，可以用它唯一标识表中每一行数据。
  类似于身份证ID

演示表单

id	name	type	season	price	date
1	葡萄	皮果	夏	27.3	2022-08-06
2	柿子	皮果	秋	6.4	2022-10-20
3	橘子	皮果	秋	11.8	2022-09-01
4	山竹	仁果	夏	40.0	2022-07-12
5	苹果	仁果	秋	12.6	2022-09-18
6	梨子	仁果	秋	13.8	2022-11-24
7	西瓜	瓜果	夏	4.5	2022-06-01
8	菠萝	瓜果	夏	11.9	2022-08-10
9	番石	瓜果	夏	8.8	2022-07-28
10	哈密瓜	瓜果	秋	7.5	2022-10-09

表 3-1：查询子句

子　句	说　明
select	查询哪些列
from	从哪个表查询
where	查找条件
group by	分组
having	分组条件
order by	排序
limit	限制行数

select语句

由(多个)子句组成

- 子句顺序　　sfwghol
  - 巧记　　**船长王哥爱喝酒**
    - sf、wg、(alco)hol
- select　　查询哪些列　　说明
  - 可以select 单行子句
    - select 1+1　　可以用于计算
    - 常搭配的算术运算　　+、-、*、/、div、%、-(取负)

      ```
      SELECT price from fruit;
      SELECT price*10 from fruit;
      ```

      price表格 37.7 54.4 21.9 50.0 22.6 23.0
  - 聚合函数(统计函数)　5大类
    - 种类　　max()、min()、sum()、avg()【求平均值】、count()【获取行数】
    - 特点　　只能用于select、order by、having这3个子句，不能用于where、group by等其他子句　　王哥不行hh　　he!需要

      ```
      SELECT SUM(price) from fruit;
      ```

      SUM(price) 144.8
    - 使用　　select sum(列名) from 表名;
    - 聚合函数的前缀
      - all　　计算所有和
      - distinct　　计算所有的非重复值的和
      - 说明　　在mysql中所有的聚合函数都可以使用一个“类型”前缀

        ```
        SELECT COUNT(DISTINCT type) 去重后的种类数,
        COUNT(type) 未去重的种类数 FROM fruit;
        ```

        去重后的种类数 3　　未去重的种类数 10
    - count
      - 使用　　select count(列名) from 表名;
      - 说明
        - 子主题
        - count(列名)计算指定列的总行数，会忽略值为 null 的行。
        - count(*)计算表的行数，不会忽略值为 null 的行。count(*) 表示包含所有的列。

          ```
          SELECT count(price) from fruit;
          ```

          count(price) 10
  - * 、`&&`、`||`、`&&` 的使用
    - 特殊列名　　必须用`括起来
      - 假如列名包含特殊字符，例如空格　　这条语句是无效的，必须也只能用英文反引号`fruit name`括起来
      - select fruit name from fruit;
    - 别名　　在实际开发中，如果使用中文别名，那么不管是在select子句中，还是在 order by 子句中建议统一不加引号，这样可以避免出错。　　因为在order中别名加引号无法识别
    - 字符串数据　　全部用`括起来

distinct　　去重操作

这是没有去重的，和前面加了DISTINCT区别显而易见了！

```
SELECT DISTINCT type from fruit;
```

type: 皮果、仁果、瓜果

1.总是紧跟在select之后，作用于整个字段
列表的所有列，而不是单独的某一列，右边是不去重的显示

2.比较的是完全相同的列，可以多列比较，但是多列必须完全相同

from　　从哪个表查询　　select子句+from子句　　说明

- 代表从table表中查找所有列数据　　select * from table
- 可以举相同的列　　select type，type from table
- *和完全列举谁好些？
  - select * from fruit
    这个本质还是下面的全查询！所以直接用下面的，更快！
  - select id,season,type,name,price,date from fruit;

  完全列举会最好，因为方式一会转化成方式二在运行，直接用方式二会更快
- 换行标准　　一个子句占一行，中间不能有空行
- 字段　　一个列，列名也叫作字段名
- 记录　　一行记录就是一行数据
- as起别名　　as关键字
  - 定义　　可以使用 as 关键字给一个列名、表名起一个别名。增强代码和查询结果的可读性。
  - 特点
    - as可以省略
    - 别名统一用`括起来
  - 常用场景
    - 列名比较长或可读性差。
    - 使用内置函数。
    - 用于多表查询。　　给表起别名
    - 需要把两个或更多的列放在一起。

where　　查询条件　　条件子句　　常常搭配一起使用的几种运算符

- 比较运算符
  - >、<、=、>=、<=、!>、!<、!=或<>、<=>
  - =与<=>的区别
    - =　　等于（不考虑null值）
    - <=>　　安全等于（考虑null值）

      ```
      SELECT 666=666 , null=null , 666<=>666 , null<=>null;
      ```

      666=666: 1　　null=null: (Null)　　666<=>666: 1　　null<=>null: 1
- 逻辑运算符　　AND、OR、NOT
  &&、||、！
- 其他运算符
  - is null、is not null、in、not in、between A and B、not between A and B
  - 运算法优先级　　NOT>AND>OR
  - 重点
    - in　　是否为列表中的值，表示从里面选
    - is null　　判断某一列的值是否为null，不允许使用=或者！=这样的运算符，而必须使用is null或is not null
      这里强调=不能被=替换

group by　　分组子句

- 定义　　对同一列进行分组
  分组统计，指的是根据“某些条件”来将数据拆分为若干组。例如有一个学生信息表，
  我们可以根据类型、性别、家乡等对学生进行分组，然后统计每个组有多少人、男女各为多少人等。　　等价于将原来的一个总查询化为在3个组里面的组内小查询了
- 说明
  - group by 子句中使用的列名一般要出现在 select子句中，注意是“一般”，而不是“一定”

    ```
    SELECT type 类型 ,
    GROUP_CONCAT(name) 该分组的具体水果
    FROM fruit
    GROUP BY type;
    ```

类型	该分组的具体水果
仁果	山竹,苹果,梨子
皮果	葡萄,柿子,橘子
瓜果	西瓜,菠萝,番石榴,哈密瓜
  - 可以使用group_concat()函数查看分组中的其他字段的所有信息
- 语法　　select 列名 from 表名 group by 列名;
- 使用
  - 单多列　　可以对单列、多列进行分组，多列显示多点，条件多点

    ```
    SELECT type 类型 ,
    count(*) 行号
    FROM fruit
    GROUP BY type;
    ```

类型	行号
仁果	3
皮果	3
瓜果	4
  - 实际例子　　Group by type表示对 type 这一列进行分组。由于 type 列的取值有3种结果，所以分为3组。

having　　分组条件

- 引出　　我们都知道，使用 group by 子句可以对列进行分组。
  如果希望通过聚合条件来选取特定的组比如取出“行数为 2”的组，应该怎样去实现呢？
  因为 where只能用于select子句，这里需要使用与group相匹配的条件子句having
- 实际使用

  ```
  SELECT type 类型 ,
  GROUP_CONCAT(name) 该分组的具体水果
  FROM fruit
  GROUP BY type
  HAVING count(*)>3;
  ```

类型	该分组的具体水果
瓜果	西瓜,菠萝,番石榴,哈密瓜
- 说明　　having子句的结果只对组内判断，如这里的count(*)>3
  这里的count就是组内查询出来的结果大于3！　　子主题

order by　　排序子句

- 语法　　select 列名
  from 表名
  order by 列名 asc或desc;
- 中文排序　　借助函数convert()实现之　　order by convert(列名using gbk);　　默认升序，如果想要降序，直接在后面加一个desc即可

  ```
  SELECT * FROM fruit ORDER BY convert(name using gbk);
  ```

name	type	season	price	date
8 菠萝	瓜果	夏	11.9	2022-08-10
10 哈密瓜	瓜果	秋	7.5	2022-10-09
3 橘子	皮果	秋	11.8	2022-09-01
6 梨子	仁果	秋	13.8	2022-11-24
- 默认是asc，升序
- 可以一升一降　　例如可以先对价格升序，再对日期降序。

  ```
  SELECT * FROM fruit ORDER BY price , date DESC;
  ```

name	type	season	price	date
7 西瓜	瓜果	夏	4.5	2022-06-01
2 柿子	皮果	秋	6.4	2022-10-20
10 哈密瓜	瓜果	秋	7.5	2022-10-09
9 番石	瓜果	夏	8.8	2022-07-28
1 葡萄	皮果	秋	11.9	2022-09-01
8 菠萝	瓜果	夏	11.9	2022-08-10

limit　　限制行数

- 语法 limit m , n
  - 代表只显示从查询结果的m数的n条结果
  - 默认是m=0

表 3-1：查询子句

子　句	说　明
select	查询哪些列
from	从哪个表查询
where	查找条件
group by	分组
having	分组条件
order by	排序
limit	限制行数

子句顺序　　sfwghol

- 巧记
  - 算法、王哥、喝酒
  - sf、wg、(alco)hol

问答题　　请简述where子句和having子句的区别？

- 作用对象不同：
  - where
    - where子句在数据分组之前起作用，用于筛选从数据库中读取的数据行。
    - 它基于表中的列进行条件过滤，决定哪些行能够进入后续的处理阶段，例如聚合计算等。
    - 作用于原始的总数据
  - having
    - having子句在数据分组之后起作用，用于筛选分组后的结果集。
    - 它通常与聚合函数一起使用，根据聚合结果进行条件过滤。
    - 作用于分组后的子数据组
- 语法及使用场景不同：
  - where
    - where子句可以使用表中的列名、常量、表达式以及比较运算符、逻辑运算符等进行条件判断。例如：WHERE column_name > 10。
    - 适用于对原始数据进行基本的筛选，比如筛选出特定日期范围内的记录、符合特定条件的客户信息等。
  - having
    - having子句通常结合聚合函数使用，例如：HAVING AVG(column_name) > 50。适用于在对数据进行分组并计算聚合值后，根据聚合结果进行进一步筛选的场景，比如找出平均销售额大于一定值的产品类别等
- 性能影响不同：
  - 一般来说，在可以使用where子句进行筛选的情况下优先使用where子句，
  - 因为它在数据分组之前进行筛选，可以减少参与分组和聚合计算的数据量，从而提高查询性能。